

LC-3 ISA Overview

ISA Overview:

The ISA is basically the interface that goes between the software programs and the computer hardware. It specifies all the information about the computer that a software programmer must be aware of, if the programmer is writing a program in the native machine language.

The ISA is required to specify a few important things. The first thing that an ISA specifies is the memory organization of the computer and its registers. The second thing the ISA specifies is the set of instructions that a computer can perform or execute. These can be arithmetic instructions like ADD and MULTIPLY, logical instructions like AND and NOT, or even control operations like JMP.

Every instruction requires certain operands to perform the operation. These operands can be represented using different formats called the data types. ISA specifies the data types that are supported by a computer for the operands in an instruction.

Finally, the operands may be stored in a storage area within the computer. The exact location of these operands in the storage area is computed based on the addressing modes. An addressing mode is, therefore, a method used to specify the location of an operand in the storage area. A computer may support many such methods and, therefore, multiple addressing modes.

LC-3 Memory Organization:

The term memory organization literally means the way the main memory of a computer is organized. Memory organization includes two key properties associated with memory: the address space and the addressability. LC-3 has an address space of 2^{16} or 65536 locations. Each location stores 16 bits, and therefore, the **addressability** of LC-3 is 16 bits or 2 bytes.

When data is transferred between a processor and the memory storage, it is done using a defined bit length called the “word.” This defined bit length is equal to the computer’s data bus, which enables one word to be transferred between processor and memory in a single operation. LC-3 has a word size of 16 bits, which is equal to the content stored in each memory location in LC-3 memory.

LC-3 Registers:

The importance of having a small amount of storage close to the ALU is to allow the results of the arithmetic and logic operations to be temporarily stored and used in subsequent instructions. We call these temporary storage locations as “Registers.” Registers can be accessed using a single clock cycle, unlike memory locations, which can take more than one clock cycle to access.

LC-3 contains a set of eight such general-purpose registers, or GPRs in short, referred to as R0, R1, R2, and so on until R7. Each GPR is uniquely identifiable and stores 1 word or 16 bits of information. Since LC-3 has eight GPRs, they can be uniquely identified using a 3-bit register number, as 2^3 is 8!

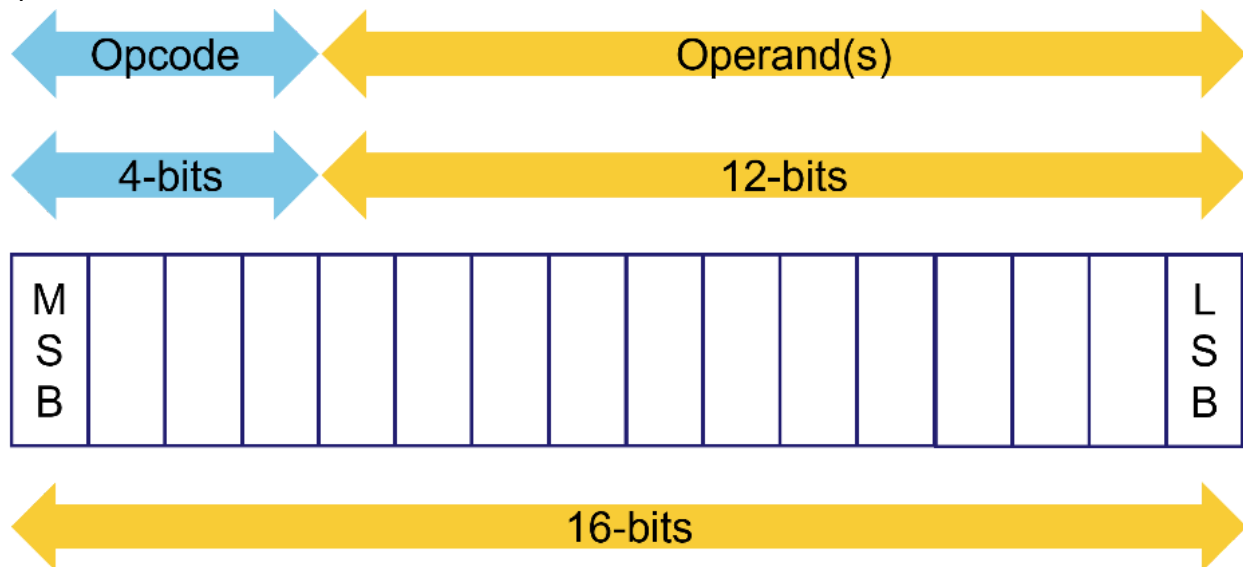
The set of eight GPRs is also called the Register Set or the Register File of LC-3.

Besides the GPR, a computer has other registers that are not directly addressable but used by or even modified during instruction processing. An example is the Program Counter, which has the address of the next instruction that is to be fetched for processing. LC-3 also has these

other registers that are required for the operations of the computer, but from the perspective of the ISA, we are only interested in the GPRs that the software programmer can use in their program. Apart from these registers, LC-3 also has three registers of 1-bit each, called the condition code registers.

LC-3 Instruction Format and Types:

LC-3 instructions are 16-bit in length and comprise two distinct parts. The first part of an LC-3 instruction consists of 4 bits, which contain the Opcode. These are the four most significant bits of the 16-bit instruction. The 4-bit Opcode specifies the operation that the instruction is meant to perform. Examples of Opcode values are 0001 for the ADD operation and 1001 for the NOT operation.



The second part of the LC-3 instruction consists of 12bits of information, which contains the operand(s). The operands are the values on which the operation specified by the Opcode is to be performed. For example, if the operation is an ADD operation, then this requires at least two operands specifying the two values that are to be added together.

There are different ways in which operands can be specified. They may be specified by referring to some memory location or some general-purpose register or may even be directly specified by including the value inside the instruction, using some of the 12bits available for the operand(s).

LC-3 instruction set contains instructions which are one of three types: operate, data movement, and control. Operate instruction types are those that “operate” on data or process the data. Data movement type instructions are those that move data—between the memory, the processing unit, and the input/output devices. Control instruction types are used to change the otherwise sequential nature of instruction execution.

LC-3 Instructions:

LC-3 instructions are summarized in the following table:

Instruction	Type	Description
ADD	Operate	Arithmetic Addition
AND	Operate	Logical Bit-wise <i>AND</i> Operation
NOT	Operate	Logical Bit-wise Complement
BR	Control	Conditional Program Branch
JMP/RET	Control	Jump to Location / Return from Sub-routine
JSR/ JSRR	Control	Jump to Sub-Routine

LD/LDI/LDR	Data Movement	Load Memory Contents
ST/STI/STR	Data Movement	Store Content to Memory
LEA	Data Movement	Load Effective Address
TRAP	Control	Invoke System Call
RTI	Control	Return from Trap / Interrupt

Opcodes for the different LC-3 instructions are provided in the following tables:

Instruction Name	OpCode
ADD	0001
AND	0101
BR	0000
JMP/RET	1100
JSR/ JSRR	0100
LD	0010
LDI	1010
LDR	0110

Instruction Name	OpCode
LEA	1110
NOT	1001
RTI	1000
ST	0011
STI	1011
STR	0111
TRAP	1111
<i>Reserved</i>	1101

LC-3 Data Types:

An LC-3 instruction stores its operand(s) using the 12 operand bits. But to understand what value these bits translate to requires knowledge of the data type of the operand. Numbers can be represented using different notations or numbering systems. For example, the same number 6 can be represented in three different ways depending on whether we use decimal notation, Roman number notation, or binary notation.

A particular representation is called a data type if there are instructions in the computer's ISA which operate on information that is encoded using that representation. As such, every ISA has its own set of supported data types and instructions that operate on those data types.

LC-3 ISA uses the 2's complement integer representation for its operand(s), and this representation is, therefore, the LC-3 ISA's supported data type.

LC-3 Addressing Modes:

An addressing mode is a method for specifying the location of an operand. In LC-3, an operand can be located in any one of the three places: the memory, a register, or as part of the instruction itself. We use the term "immediate operand" to refer to such an operand that is present as part of the instruction. Sometimes, the immediate operand is also referred to as a "literal" operand since the operand is literally formed out of the operand bits of the instruction. Based on the location of the operand in LC-3, five different addressing modes are supported in LC-3 ISA. When the operand is stored in the memory, three addressing modes are possible, namely PC-Relative, Indirect, and Base+Offset. The other two addressing modes are the Register and Immediate (or literal) addressing modes, corresponding to the other two possible locations of an operand in LC-3.

Different types of instructions use different addressing modes. While operate type instructions use the Register and Immediate addressing modes, data movement type instructions make use of four addressing modes: PC-Relative, Indirect, Base+Offset, and Immediate.

The PC-Relative, Indirect, and Base+Offset addressing modes use different methods to provide the address of the memory location that the instruction needs to use. We refer to the address of this memory location as the effective address. The effective address is computed differently for each of these addressing modes.

In the PC-Relative addressing mode, an offset value is included as part of the instruction operand bits. Since the data type of this offset value is 2's complement, the offset value is first sign-extended before it is added to the Program Counter. This gives us an effective address.

In the Indirect addressing mode, a memory address is first computed in the same manner as done with PC-relative addressing mode, i.e., by using the offset and adding it to the program counter after sign extension. However, the difference stems from the fact that this address is **not** the effective address we want. Rather, this address points to a memory location, which contains the effective address. Notice the indirection—we are using the first address computed to read a memory location, which has the effective address. It is due to this indirection that the addressing mode is called Indirect.

Lastly, in the Base+Offset addressing mode, the instruction operand bits contain two things—the ID of a Base Register and an Offset. Similar to the PC-Relative addressing mode, the offset is first sign-extended. However, instead of adding it to the Program Counter, it is instead added to the Base Register. This gives us an effective address.

Notice the similarity between the PC-Relative and the Base+Offset addressing modes. The computation of the effective address follows similar steps, except that the registers used are different—the program counter in one and the base register in the other.

LC-3 Condition Code Registers:

LC-3 has three single-bit condition code registers called N, Z, and P. These stand for negative, zero, and positive, respectively. Each time an operate or load instruction updates a general-purpose register, the condition code registers are set or reset. An operate instruction performs an arithmetic or logic computation and writes a result to a GPR. On the other hand, a load instruction reads from memory and updates it in the GPR. Depending on whether the result written to the GPR is negative, zero, or positive, the N, Z, or P condition code register is set,

respectively. Note that exactly one of the N, Z, and P registers will be set, whereas the others will be reset. If the result is negative, the N register is set, and Z and P are cleared. If the result is zero, Z is set and N and P are cleared. If the result is positive, P is set and N and Z are cleared. Condition code registers are used to check for specific conditions that can be used to alter the sequence of execution of the instructions in a software program.

LC-3 Instruction Set

LC-3 Operate Instructions

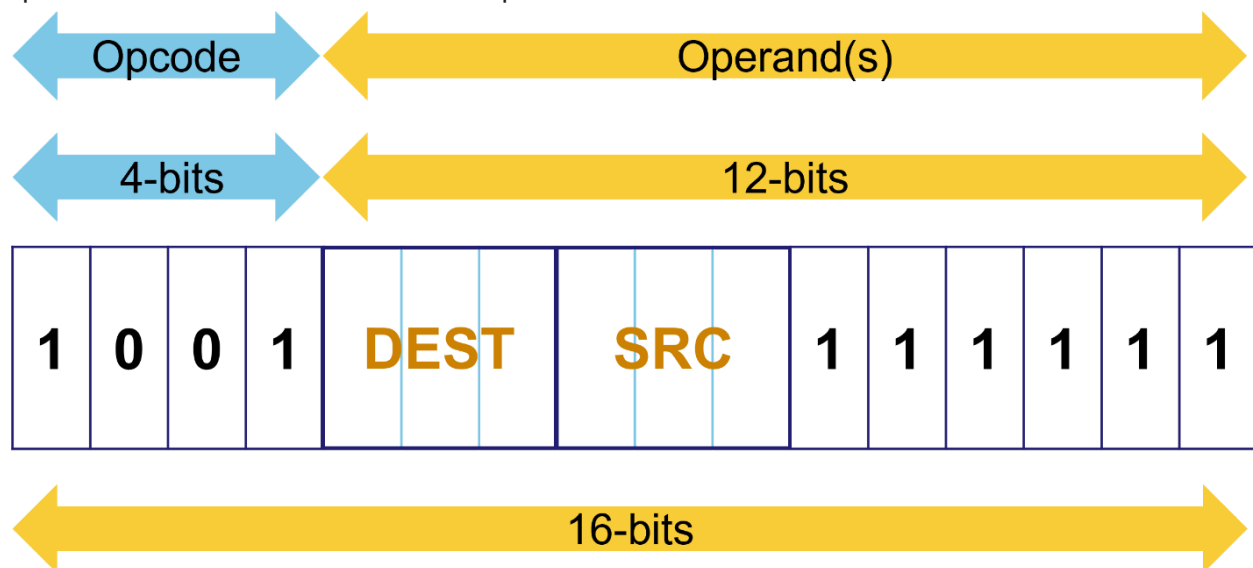
The three LC-3 operate instructions—NOT, AND, and ADD are identified using their unique 4-bit Opcodes. The remainder 12 bits of the instruction are the operand bits.

The NOT instruction is a unary operation, which means that it takes one source operand and performs the complement operation on it. It then stores the result in a register identified by the destination operand.

AND and ADD instructions, on the other hand, involve two source operands, on which the operation of bitwise and the operation of arithmetic addition is performed, respectively. The result is once again stored in a destination register identified by the destination operand. The source and destination operands are specified using some of the 12 operand bits.

LC-3 NOT Instruction

The diagram shown below depicts how the operand bits are used for the NOT instruction. Since NOT is a unary operation, only one source operand is required. NOT instruction uses the Register addressing mode for both the source and destination operands. Hence, three bits each are required for specifying the source register and the destination register. This consumes 6 of the 12 available operand bits. The remainder of the six operand bits are fixed at bit value 1.

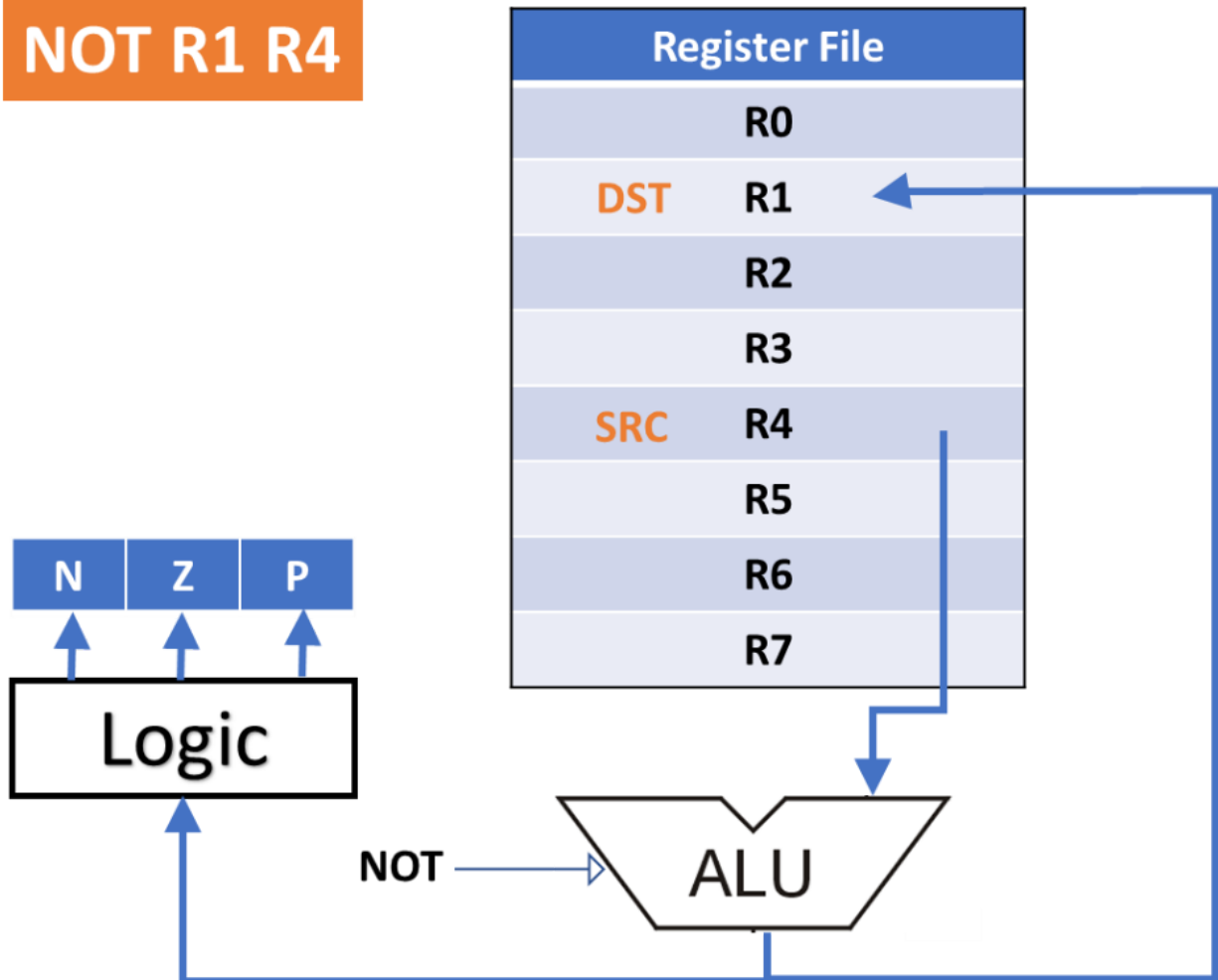


The diagram shown below depicts the data path involved in the execution of the NOT instruction.

The ALU contained in the processing unit of the LC-3 shall perform this operation. The source operand for the ALU is register R4, as shown in the diagram. The ALU knows that it needs to perform the NOT operation (based on the control signal it receives from the control unit). The result of this operation gets stored in the destination operand, which is register R1 in this diagram. Also, the condition code registers are set or reset based on the result of this operation.

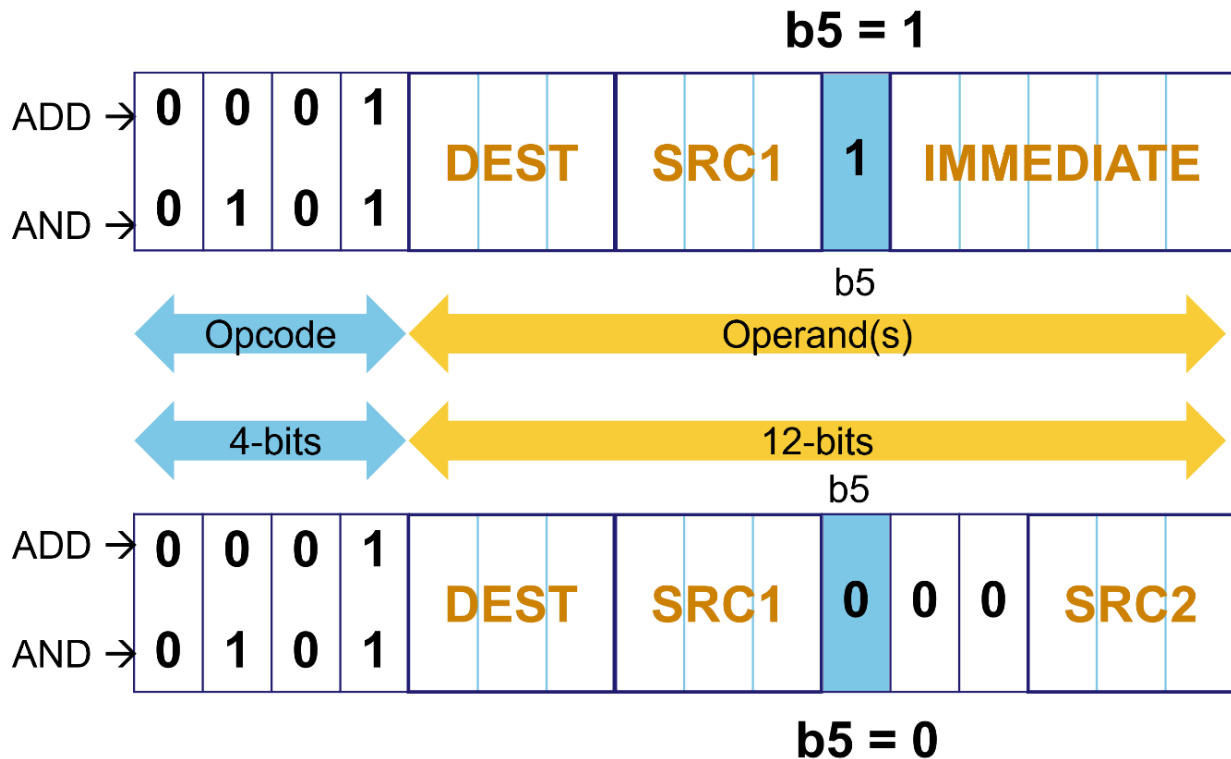
Note that while the source and destination registers are distinct in this example, the two of them can also be the same. In such a case, the source and destination operand bits contain the identity of the same register.

NOT R1 R4



LC-3 ADD/AND Instructions

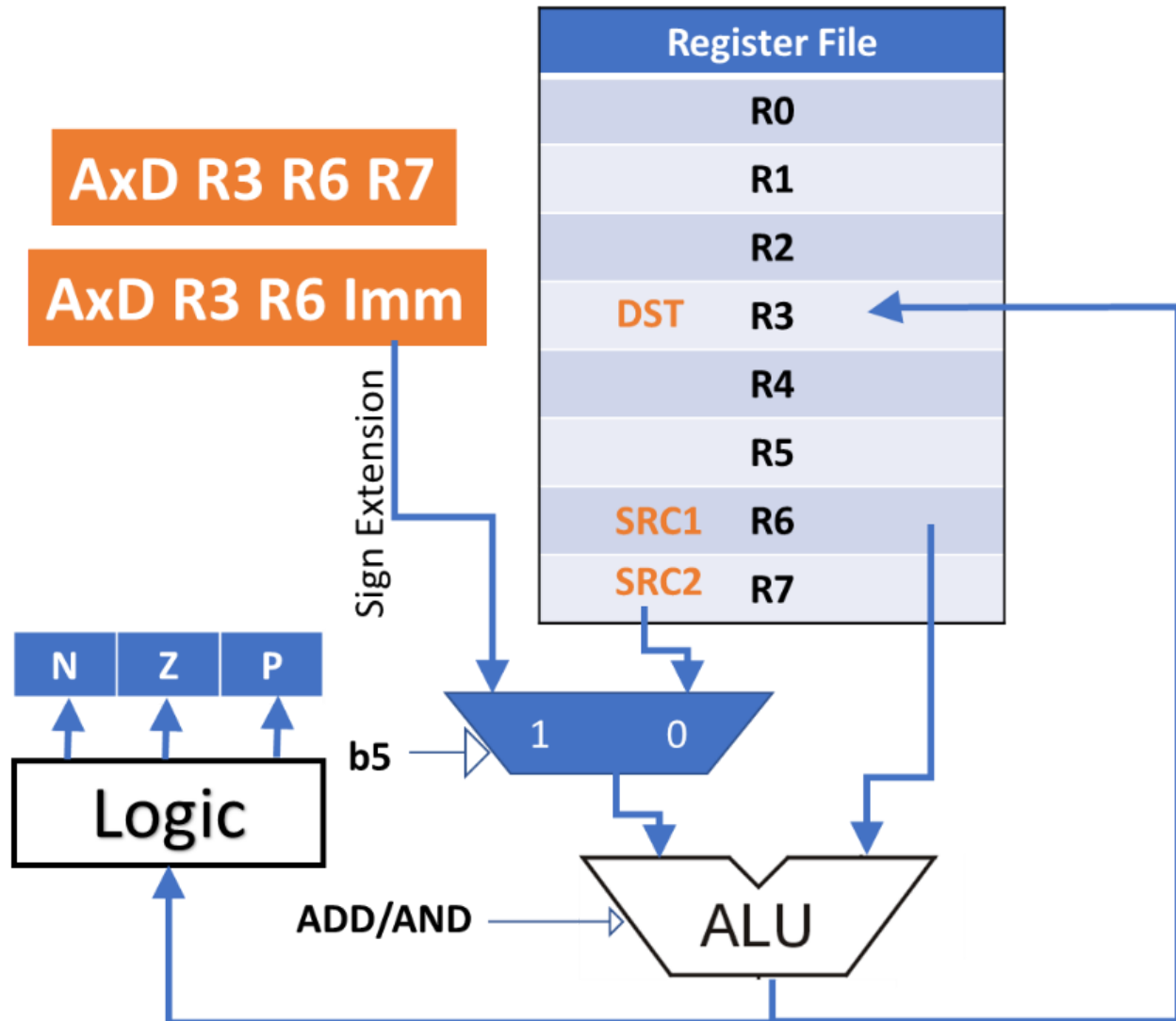
The diagram here depicts how the operand bits are used for ADD and AND instructions. Since both these instructions perform binary operations, both require two source operands and have the same format for the operand bits.



The destination operand and one of the source operands are always specified using the Register addressing mode and occupy the same operand bit positions as in the NOT instruction. However, the specification for the second source operand differs based on whether Immediate addressing mode or Register addressing mode is used to specify it.

The b5 bit is used to differentiate and indicate whether the immediate or register mode is used for specifying the second source operand. If the b5 bit is 1, the second source operand is specified using the remaining five bits using immediate addressing mode. However, if the b5 bit is 0, then the second source operand is specified using the three LSB bits, namely b0, b1, and b2, using the Register addressing mode. The remaining unused operand bits, namely b3 and b4 are fixed to value 0.

The diagram shown below depicts the data path involved in the execution of the ADD or AND instruction.



While the depiction of the data path in this diagram may seem a bit complicated, in reality, it is not much different than the NOT instruction data path. The only additional device added to the data path is a MUX, which helps to select the second operand to the ALU, based on the control bit b5. The other parts of the data path, including the register file, the N,Z, and P condition code registers, and the ALU itself, are the same as in the NOT instruction data path. The additional MUX device helps to select between the immediate operand from the instruction itself or the register from the register file, based on whether bit b5 is 1 or 0, respectively. In the case of the immediate addressing mode for the second operand, the bits b0 to b4 are first sign-extended to 16 bits before they are provided as one of the two inputs to the MUX.

Note again that the source and destination registers are distinct in this example. However, the destination register could also be the same as one of the source registers involved in the operation.

LC-3 Data Movement Instructions

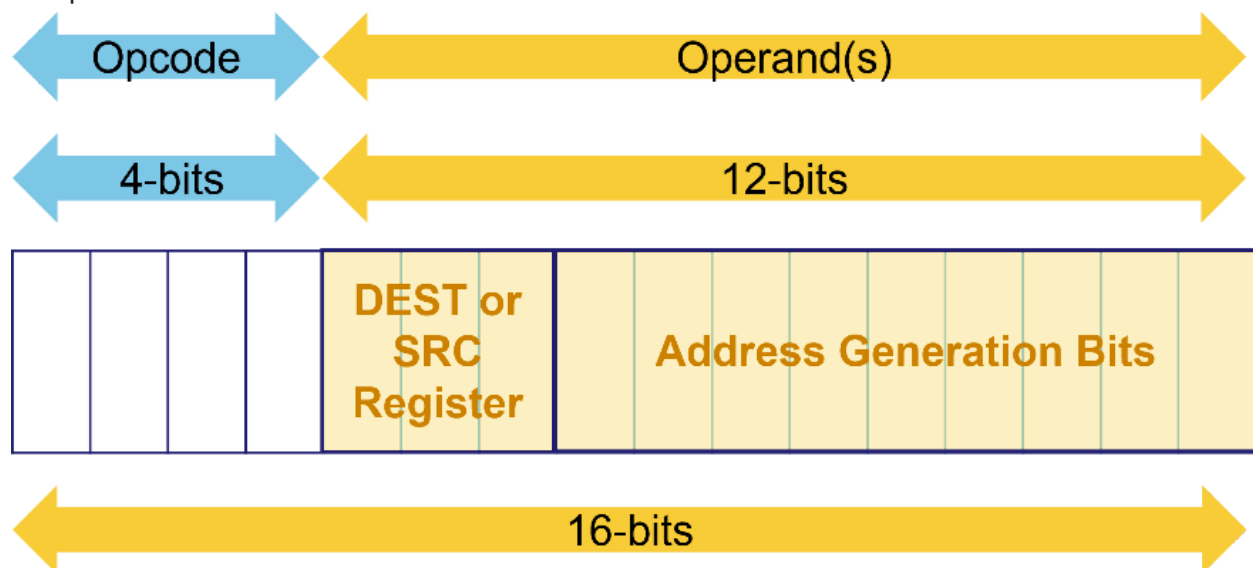
Data movement instructions are used for moving data to and from general-purpose registers and memory. A load instruction loads information from memory to a register, whereas the store instruction stores information from a register to memory. Data movement instructions are also used for moving data between registers and input/output devices. The load and the store instructions have three variants each, which use the three different addressing modes.

The Load Effective Address (or the LEA) instruction is not really a data movement instruction, and it is debatable which category to put it into. It is not an operate instruction since it does not perform any arithmetic or logical operation. Neither is it a control operation since it does not control the program execution sequence. For want of a better place, the LEA instruction is covered as part of the data movement instructions.

The different LC-3 data movement instructions are identified using their unique 4-bit Opcodes. The instructions specify one destination operand and one source operand using the 12 operand bits in the instruction.

In the load and store instruction variants, LD/LDI/LDR and ST/STI/STR, the information contained in the location specified by the source operand gets copied into the location specified by the destination operand. The content of the source location do not change; only the content in the destination location is updated.

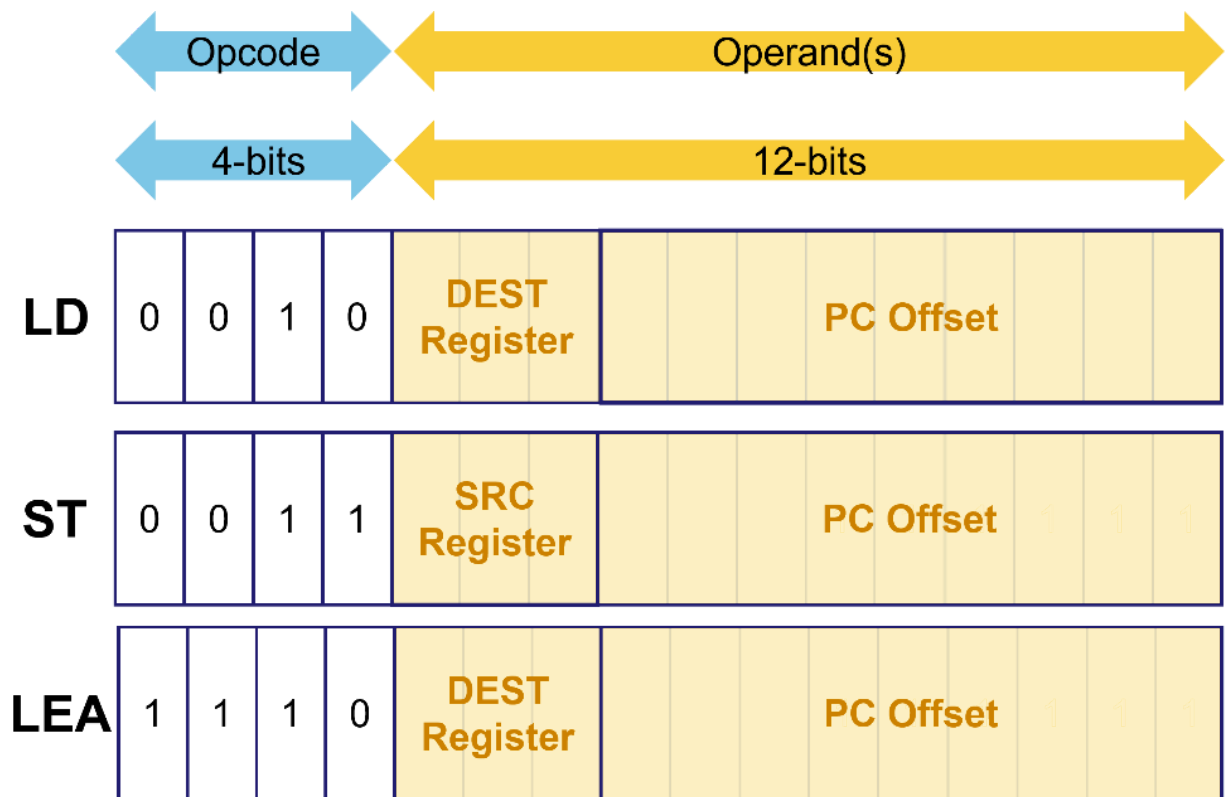
The diagram shown below depicts the general format for the data movement instructions and how the operand bits are used.



The first four MSB bits are, of course, for the Opcode. The next three bits specify the destination register or the source register. In the case of the load instructions, this shall be the destination register, into which, the value from the source location is to be loaded or copied. In the case of the store instructions, this shall be the source register, which will be used to update the content of the destination location. The remainder of the operand bits are used as “address generation bits.” These bits are used to generate the address of the location specified by the second operand. The mechanism to use these bits for address generation is dependent on the addressing modes: PC-Relative, Indirect, or Base+Offset.

LD/ST/LEA Instructions

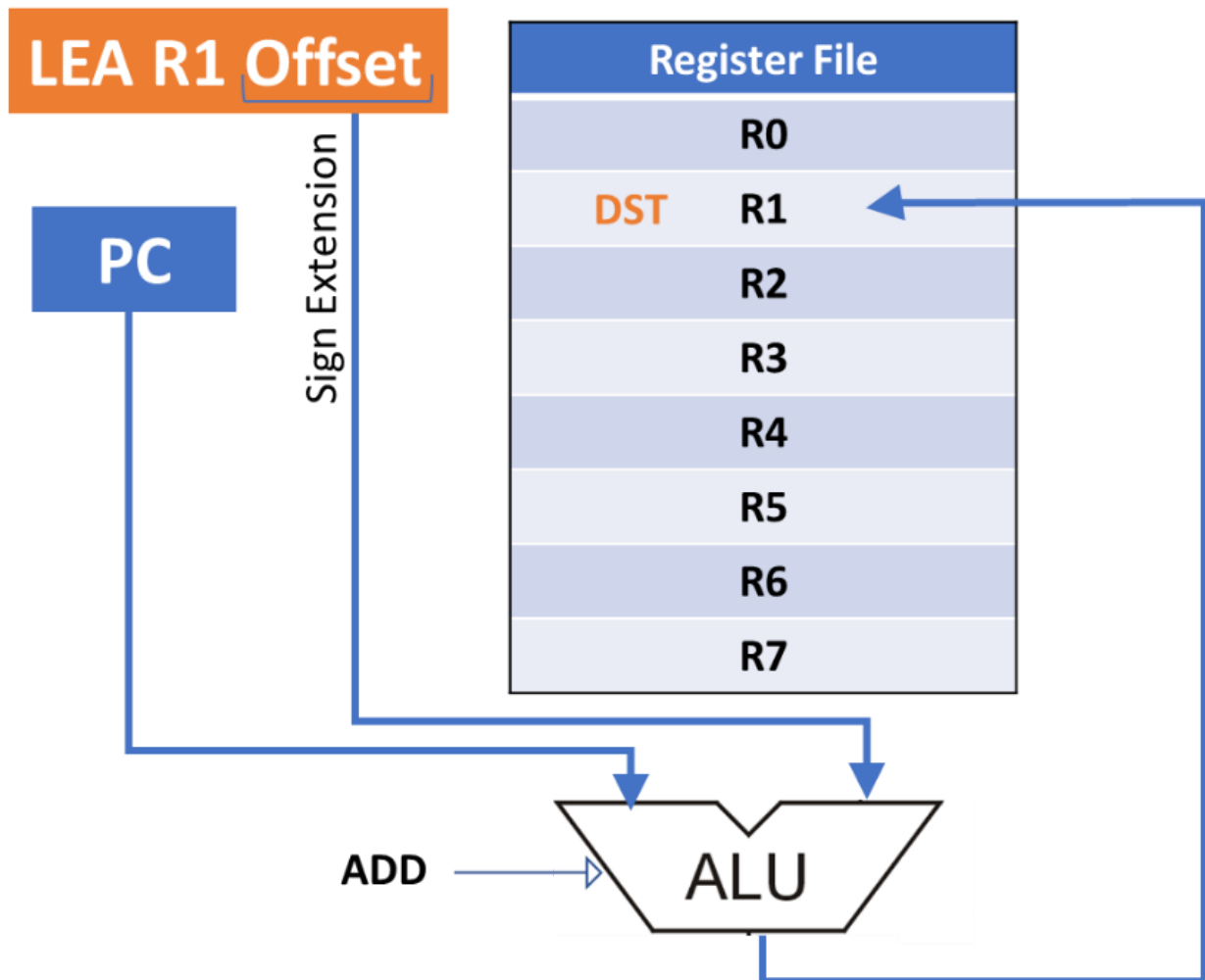
The diagram shown below depicts the format for the LD, ST, and LEA instructions, all of which use the PC-Relative addressing mode. In the case of the LD and LEA instructions, which perform the load operations, the first three MSB operand bits specify the destination register. For the ST instruction case, which performs the store operation, these bits are instead specifying the source register. The remaining operand bits are used to specify the PC Offset to be used with the PC-Relative addressing mode to get the location of the second operand.



As discussed in the section on LC-3 data types and addressing modes, a PC offset value uses a 2's complement data type. Thus, the offset value is first sign-extended before it is added to the Program Counter to give us the effective address of the second operand.

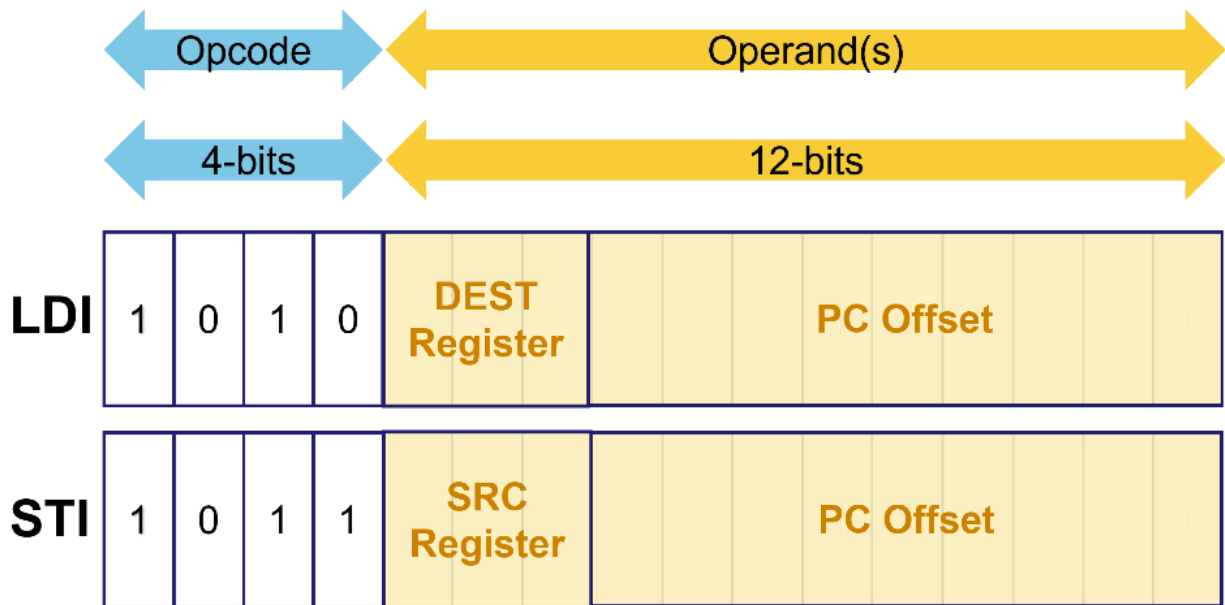
The diagram shown below depicts the data path involved in the execution of the LD or the load instruction. The first step in the data path involves the computation of the effective address of the source location from where the data is to be loaded into the destination register. Since LD uses PC-Relative addressing mode, the effective address of the source location is computed by adding the incremented program counter with the sign-extended PC-Offset specified in the instruction. The ALU performs this addition operation, where the two inputs to the ALU are the incremented PC and the sign-extended value obtained from the offset. The output from the ALU is the computed effective address of the source location. Next, the computed effective address of the source location is fed to the memory address register. The data from this memory location is then copied to the memory data register. Finally, this data is copied from the MDR to the destination register specified in the LD instruction, in this case, R1. Also, the MDR triggers the logic that sets the condition codes of the LC-3.

loads the specified destination register with the effective address computed by adding the incremented program counter with the sign-extended PC-Offset specified in the instruction. The output from the ALU, which is the computed effective address, gets stored in the destination register—in this diagram, R1. This is how the load-effective address instruction is executed.



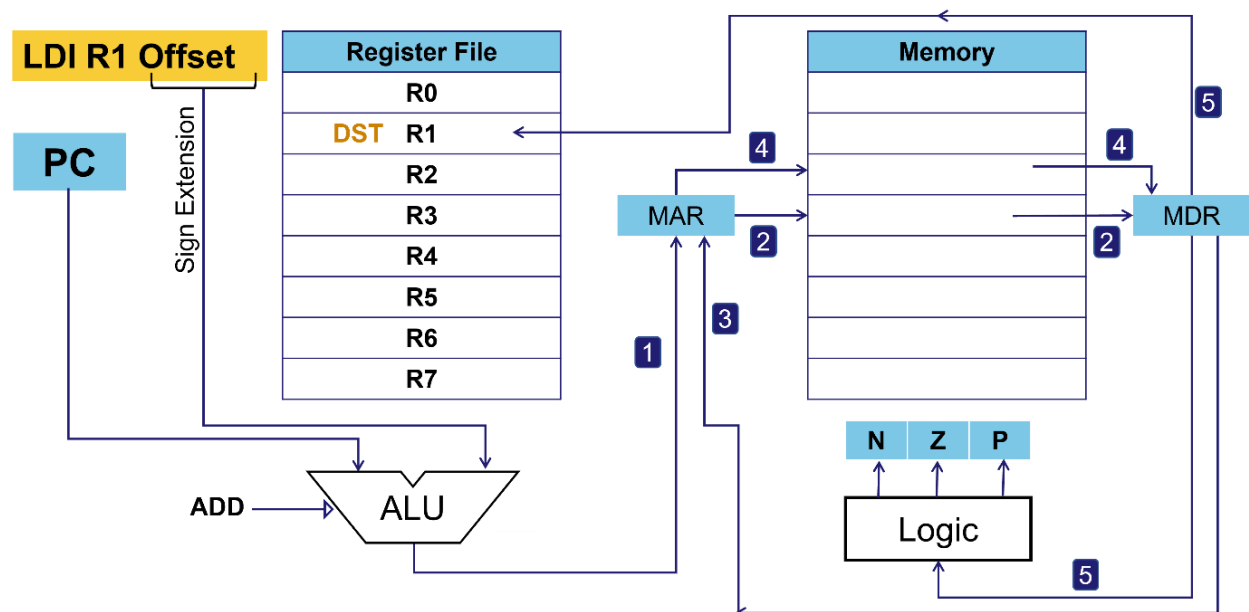
LDI/STI Instructions

Since the PC-Relative addressing mode is dependent on the value of the program counter, only addresses that are within 256 words of the current instruction can be addressed with the PC-relative addressing mode. So, to address the other word locations, either the Indirect addressing mode or the Base+Register addressing mode can be used. The diagram shown below depicts the format for the LDI and STI instructions, which use the Indirect addressing mode.



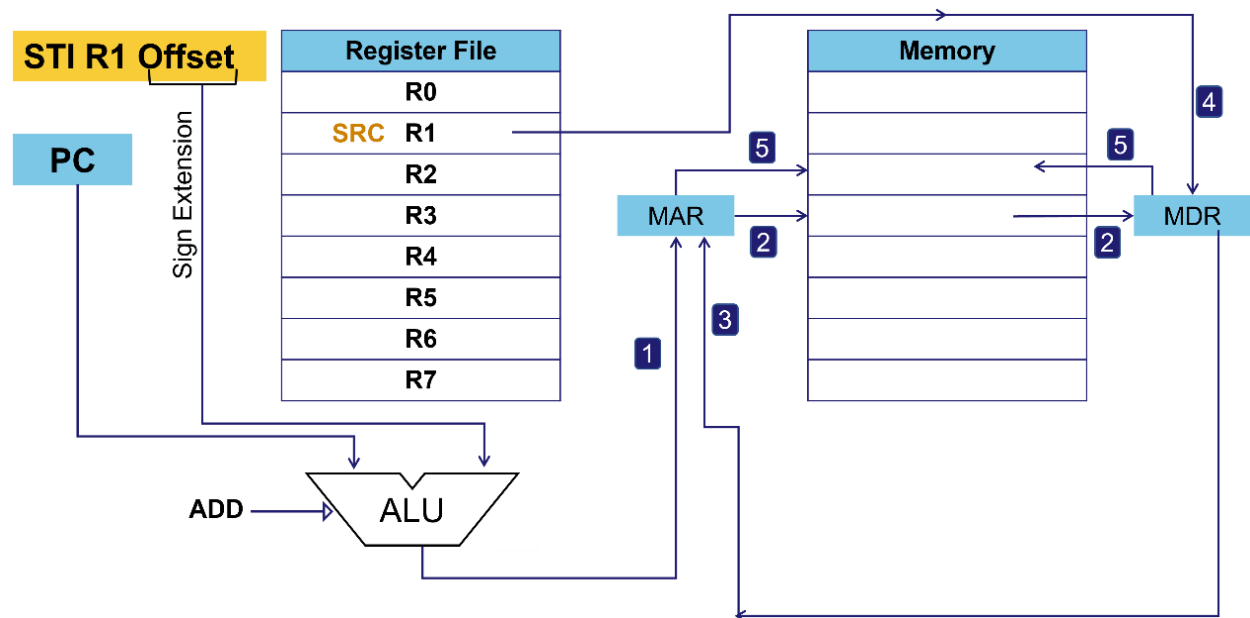
In the case of the LDI instruction, the first three MSB operand bits specify the destination register, just like in the LD instruction. Similarly, for the STI instruction, these bits are instead specifying the source register, like in the case of the ST instruction. The remaining operand bits are used to specify the PC Offset to be used with the Indirect addressing mode to get the location of the second operand.

The diagram shown below depicts the data path for the LDI Instruction.



The first few steps of the LDI instruction are similar to the LD instruction. An effective address is computed by adding the incremented program counter with the sign-extended PC-Offset specified in the instruction. The ALU performs the addition operation to obtain a computed effective address. This computed effective address is fed to the Memory Address Register. The data from this memory location is then copied to the Memory Data Register. Till here, the steps are the same as in the LD Instruction.

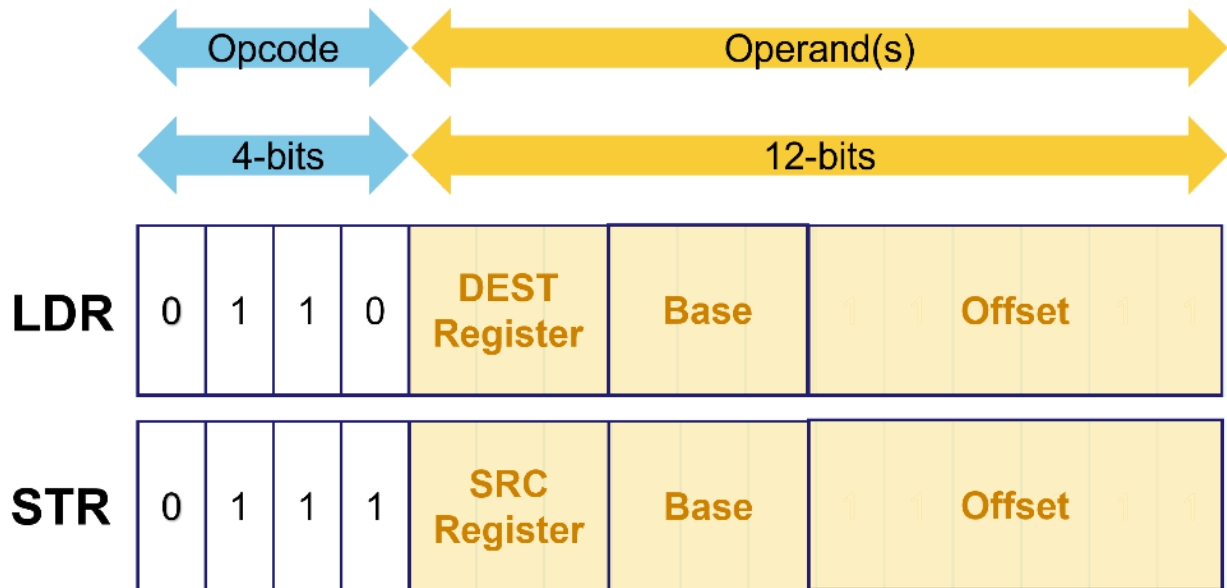
Now, here is where the difference due to the Indirect mode comes in. The data from the memory data register is copied back to the memory address register to implement the indirection. The memory address registers reference another memory location this time, the data from which is again copied into the memory data register. Now, the steps from the load instruction are executed—the data is copied from the MDR to the destination register specified in the LDI instruction, in this case, R1. Then, the logic that sets the condition codes of the LC-3 is also triggered. The diagram shown below depicts the data path for the STI instruction.



The first few steps of the STI instruction are similar to that of the LDI instruction. An effective address is computed by adding the incremented program counter with the sign-extended PC-Offset specified in the instruction. The ALU performs the addition operation to obtain a computed effective address. This computed effective address is fed to the Memory Address Register. The data from this memory location is then copied to the Memory Data Register. Then, the data from the memory data register is copied back to the memory address register to implement the indirection. Next, the steps related to the store instruction are executed. Data from the source register, here R1, is copied into the memory data register. The memory address register now references the destination memory location where data from the memory data register is copied into. This completes the indirect store procedure.

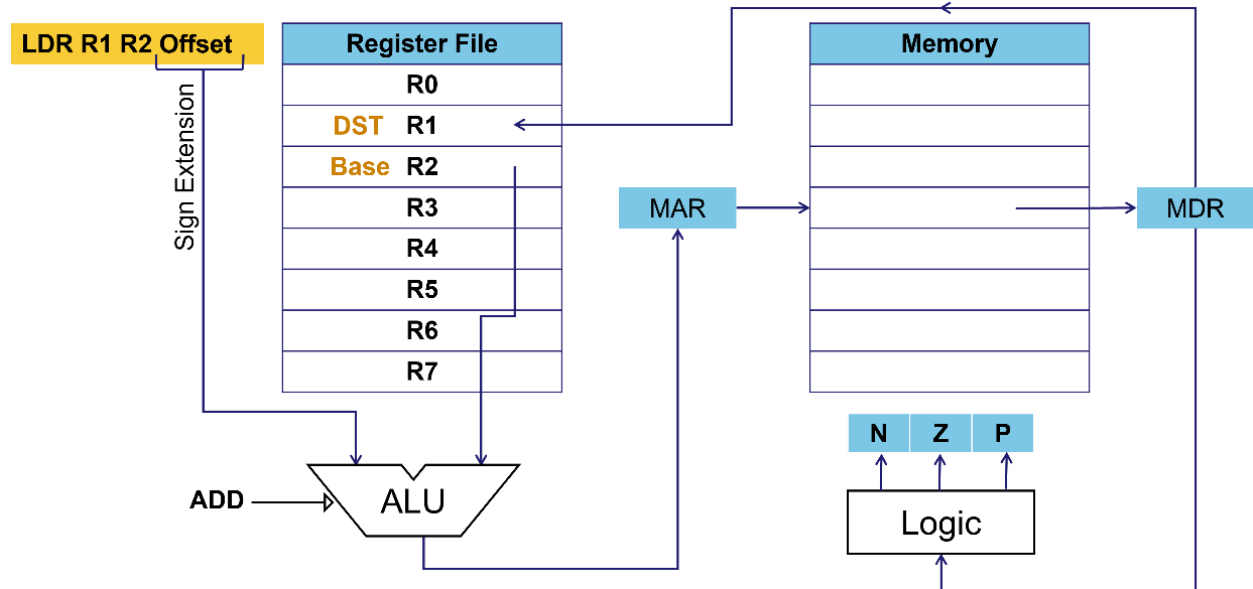
LDR/STR Instructions

Let us discuss the LDR and STR instructions, which use the Base+Offset addressing mode.



In the case of the LDR instruction, the first three MSB operand bits specify the destination register, just like in the LD instruction. Similarly, for the STR instruction, these bits are instead specifying the source register, like in the case of the ST instruction. The next three bits are used to specify the Base register, whereas the remaining six operand bits are used to specify the Offset to be used with the Base address to get the location of the second operand. Let us look at how the data paths work for the LDR and the STR instructions.

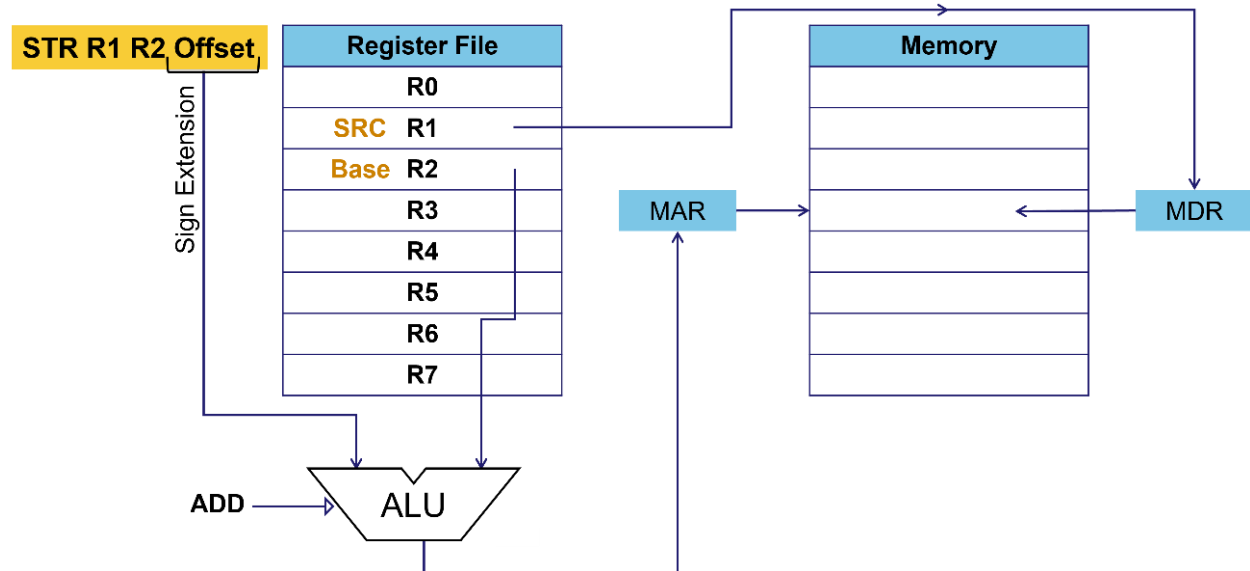
The diagram shown below depicts the LDR data path.



The first step in the data path involves the computation of the effective address of the source location from where the data is to be loaded into the destination register. Since LDR uses Base+Offset addressing mode, the effective address of the source location is computed by adding the base address available in the base register, here R2, with the sign-extended Offset value specified in the instruction. The ALU performs this addition operation and gives us the computed effective address of the source location.

Next, the computed effective address of the source location is fed to the Memory Address Register. The data from this memory location is then copied to the Memory Data Register. Finally, this data is copied from the MDR to the destination register specified in the LD instruction, in this case, R1. Also, the MDR triggers the logic that sets the condition codes of the LC-3.

The diagram shown below depicts the data path for the STR instruction.



The computation of the effective address is similar to the LDR instruction. The effective address of the destination memory location is computed by adding the base address available in the base register, here R2, with the sign-extended Offset value specified in the instruction. The ALU performs this addition operation and gives us the computed effective address of the destination location. Next, the computed effective address of the destination location is fed to the Memory Address Register. The data is copied from the source register (here R1) to the memory data register. Finally, the data is copied from the Memory Data Register into the memory location pointed to by the Memory Address Register. This completes the execution of the STR instruction.

LC-3 Control Instructions

Control instructions are used to change the sequence of execution of instructions in a software program. Without control instructions, all instructions in a program will execute in sequence, one after the other. There are cases where we require out-of-sequence execution, where control instructions come in handy.

Control instructions fall under the following category:

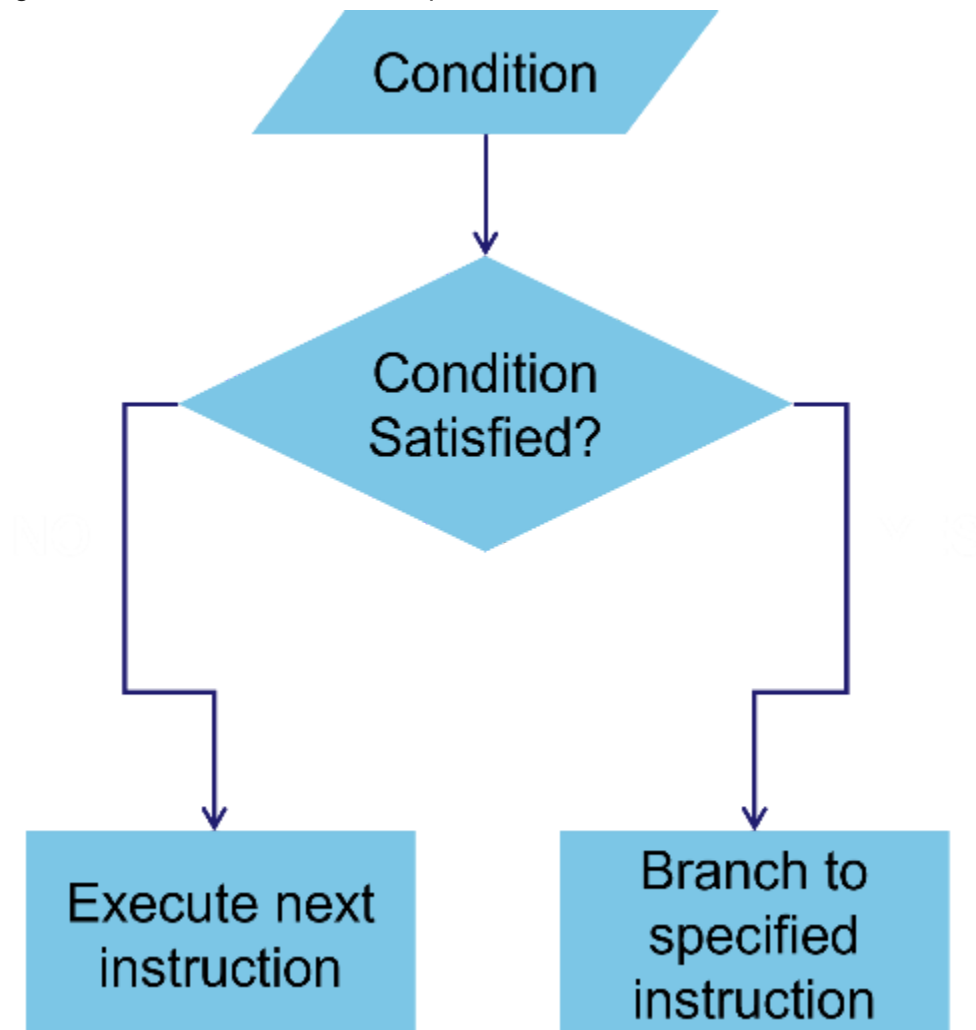
- Conditional branch instruction, which is called BR (short for branch) in LC-3
- Unconditional branch instruction, which is called JMP or Jump in LC-3
- Branch to a system call implemented by the operating system, done via the TRAP instruction

LC-3 has some other control instructions used for invocation and return from sub-routines. This last category includes the rest of the LC-3 control instructions, namely the JSR and JSRR (used for Jump to a subroutine), RET (to return back to original execution from a sub-routine), and RTI (to return back from a trap or interrupt).

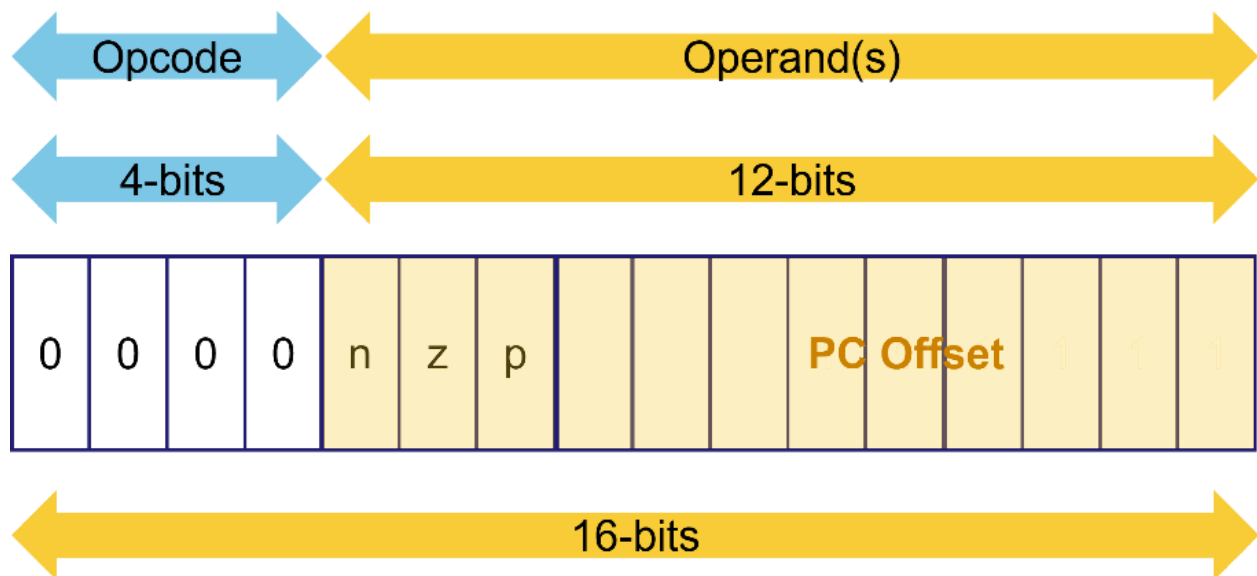
In this module, the first three categories of control instructions are covered, which are probably the most often used and simplest of control instructions. The other instructions will be covered when discussing sub-routines in later modules.

Branch Instruction

At the heart of the BR instruction is a method to specify a condition that is to be evaluated. The processor evaluates this condition specified in the BR instruction. If the condition fails, the software program continues normal sequential execution. However, if the condition is met, the software program branches to an instruction specified in the BR instruction.



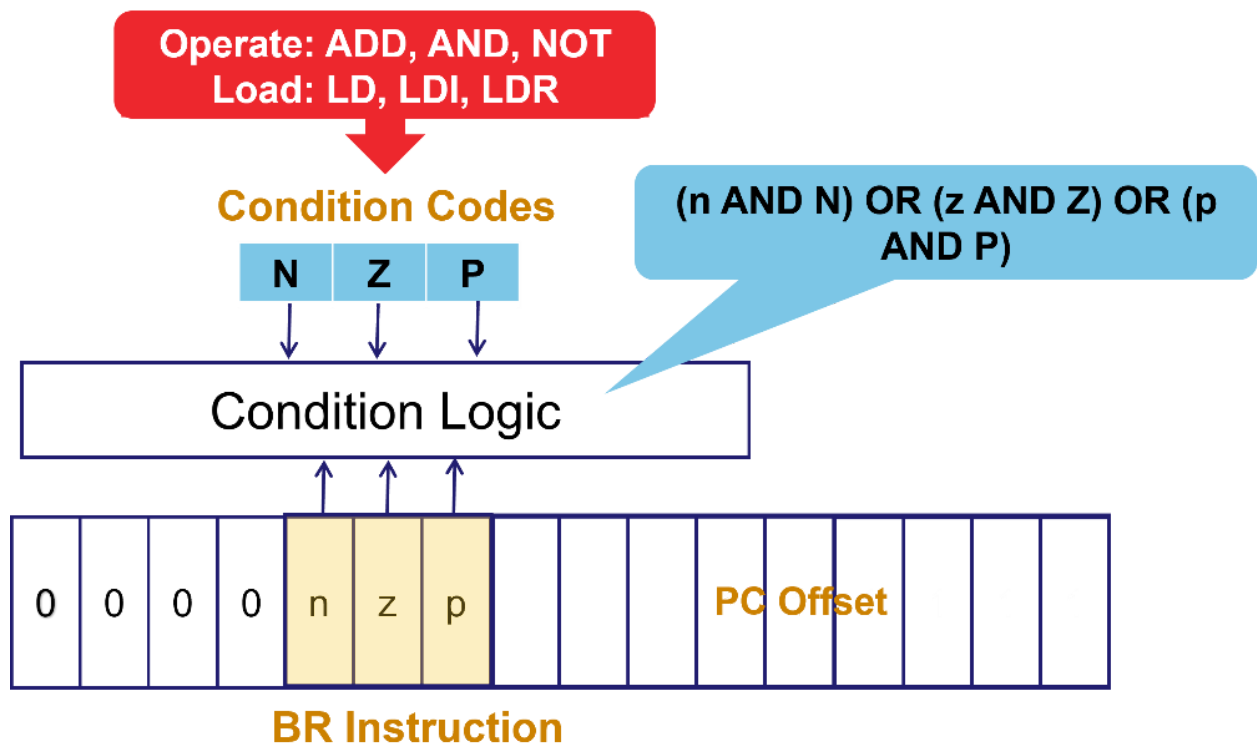
The diagram shown below depicts the format for the BR instruction and how the operand bits are used. The first four MSB bits are for Opcode 0000. The next three bits are the n, z, and p flag bits. These flag bits correspond to each of the three condition code registers. As we will shortly see, these flag bits are used to specify the condition for executing a branch in the software program. The remainder of the operand bits are the PC Offset bits. Since the BR instruction uses the PC-Relative addressing mode, this offset is used to compute the effective address of the instruction to be executed in case the specified condition is met.



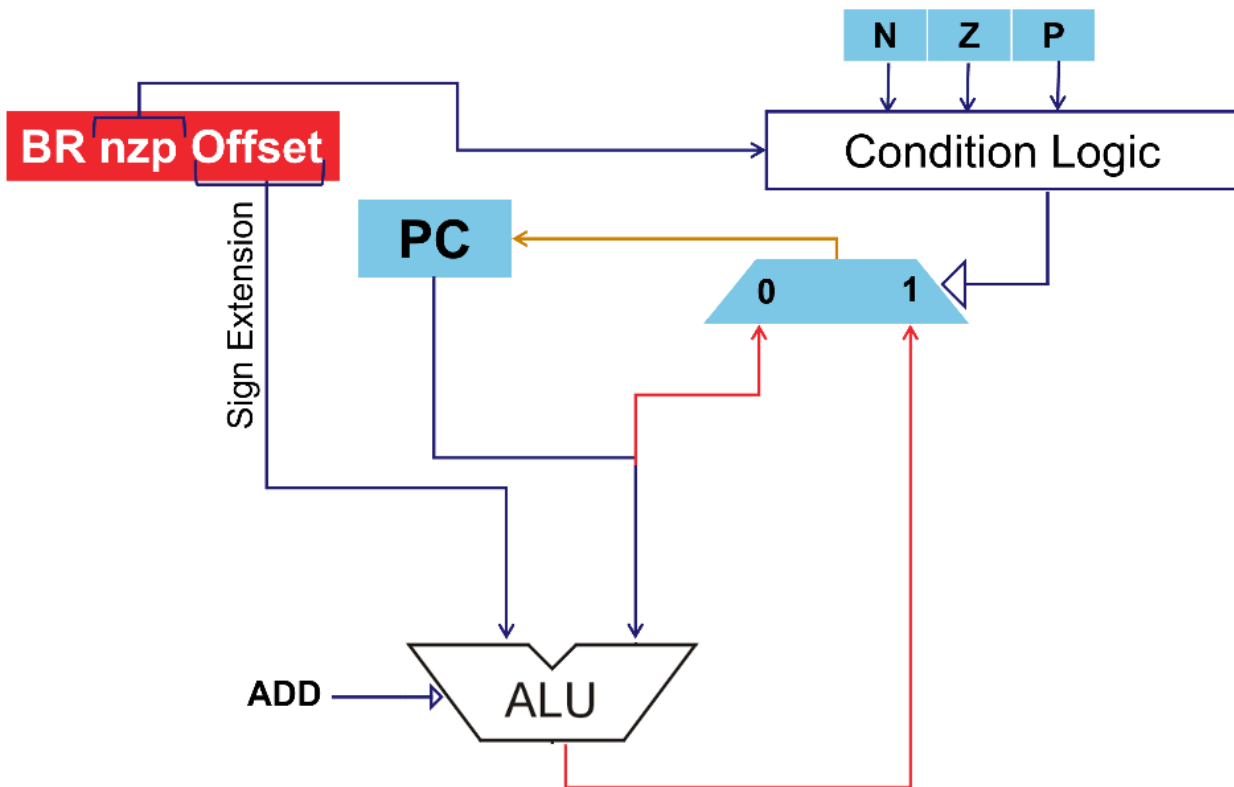
So, how do we specify the condition using the n, z, and p flag bits in the BR instruction? What is the condition that we are checking?

Recollect that the three operate instructions (ADD, AND, and NOT) and the three load instructions (LD, LDI, and LDR) set the three condition code registers (capital) N, (capital) Z, and (capital) P. The condition codes are set depending upon whether the value written by these instructions into the general-purpose registers is negative, zero, or positive, respectively.

The BR instruction is used to execute a conditional branch depending on the result of these operate or load instructions. Inside the processor, a condition logic evaluates the condition specified in the BR instruction. This condition logic implements the logical operation depicted in the diagram.



The diagram shown below depicts the BR instruction data path.



The first step in the data path involves the computation of the effective address of the instruction that would be executed in case the branch condition were to be met. Since BR uses PC-Relative addressing mode, the effective address of the instruction is computed by adding the incremented program counter with the sign-extended PC-Offset specified in the instruction. The ALU performs this addition operation to give us the computed effective address of the next possible instruction. The computed effective address becomes one of the inputs to the selection MUX. What should be the other input to the MUX?

If the BR condition fails, we wish to continue the program execution sequentially and not execute the branch. The other input to the selection MUX would, therefore, be the incremented program counter itself.

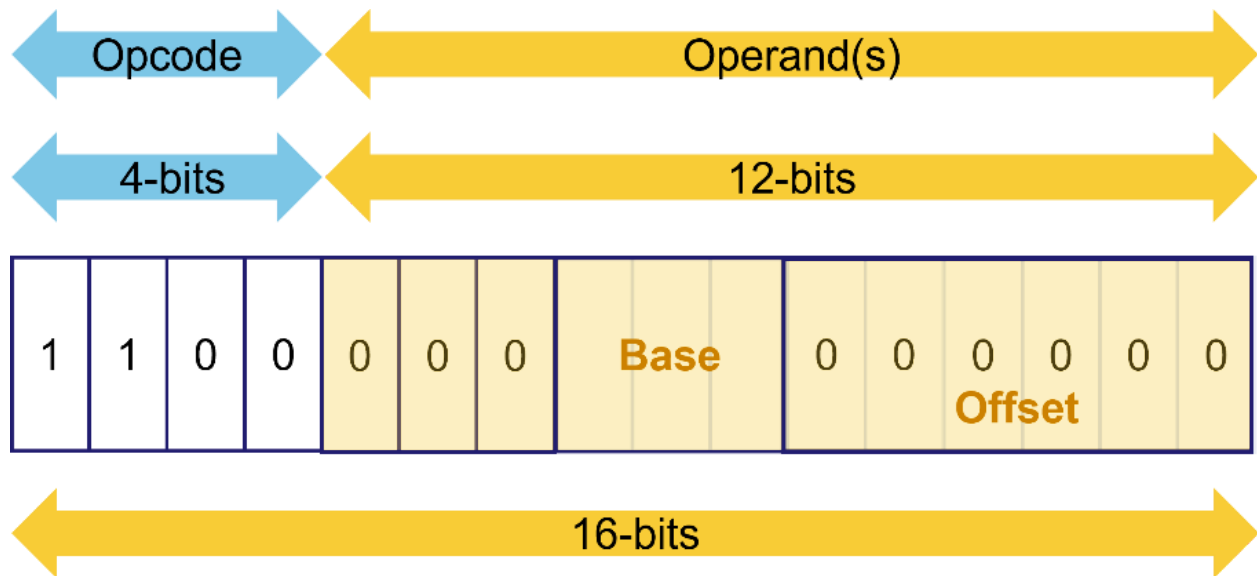
The control signal to the MUX will come from the BR condition logic. As just discussed, this depends on the n, z, and p bits in the BR instruction and the value of the condition codes. The output from the MUX overwrites the program counter. This is the address of the next instruction to be executed.

Jump Instruction

The diagram below depicts the format for the JMP instruction and how the Operand Bits are used.

The first four MSB bits are for the Opcode 1100. The next three bits are unused and are set at 0. The following three Operand bits specify the Base register. Since the JMP instruction uses the Base+Offset addressing mode, the value in the Base Register is used to compute the effective address of the next instruction to be executed. The last five operand bits are the Offset value, except that the offset in the JMP instruction is all zeros.

Recollect that an unconditional branch could also be implemented using the BR instruction by setting the n, z, and p flag bits to be all 1s in the BR instruction. However, since the BR uses the PC-Relative addressing mode, the target addresses to which a branch can be executed are limited. The JMP instruction overcomes this limitation.



The diagram shown below depicts the data path involved in the execution of the JMP instruction. There is nothing much happening here and is quite straightforward. The value stored in the base register is simply copied into the program counter, making the software program to branch to the instruction specified by the address in the base register.

JMP R1

Register File	
	R0
Base	R1
	R2
	R3
	R4
	R5
	R6
	R7

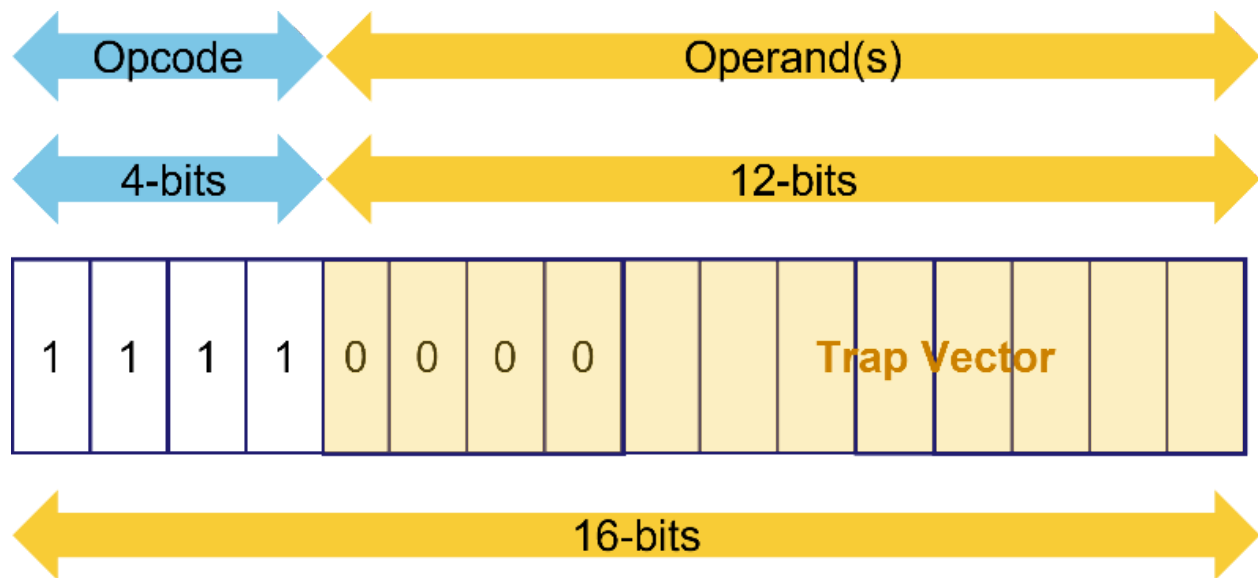
PC

TRAP Instruction

The TRAP instruction is used to transfer control to the operating system. It is used when we want the operating system to perform a task that the application code is not allowed to perform. Such tasks are implemented in the operating system as service routines.

The diagram here depicts the format for the TRAP instruction and how the operand bits are used.

The first four most significant operand bits are unused and are set at 0. The following eight operand bits specify the Trap Vector. The trap vector is a unique code that is used to specify the invoked service routine.



The table shown below provides the trap vector values for some of the commonly used service routines that are invoked from the application code. The first two are used for I/O operations, whereas the last one is used to halt the execution of the processor. For I/O operations, the register 0 or R0 is used to store the data that is input to the computer from an input device and also to store the data that is to be output to an output device.

Trap Vector	Service Routine
x23	Input a character from the keyboard
x21	Output a character to the monitor
x25	Halt the processor